Memory and Long-Context Mechanisms for Long-Horizon LLM Agents

Abstract

The transition of Large Language Models (LLMs) from reactive, stateless text predictors into autonomous, self-evolving agents fundamentally relies on their ability to maintain, organize, and utilize context over extended temporal horizons. As agents are deployed in multi-turn dialogues, personalized assistance, complex planning scenarios, and unbounded interactive environments, the constraints of finite context windows become severe architectural bottlenecks. This comprehensive survey provides an exhaustive examination of memory and long-context mechanisms in modern LLM-based agents. The analysis bridges the foundational architectural layer—encompassing positional encoding extrapolations, Key-Value (KV) cache compression, and state-space models—with the cognitive agentic layer, where memory is formalized as an active, policy-driven write-manage-read loop. By synthesizing the recent academic literature, this report establishes a unified taxonomy that categorizes memory representations into token-level inputs, intermediate latent states, and parametric weights. Furthermore, the discussion traces the evolutionary trajectory of agent memory from raw trajectory storage to reflective semantic filtering and, ultimately, cross-trajectory experiential abstraction. Specialized architectures, including neuro-symbolic stores, code-based state representations, and decentralized multi-agent semantic protocols, are critically evaluated alongside emerging multi-session benchmarking frameworks. The synthesis reveals that effective long-horizon operation requires not merely expanding the observable context window, but implementing rigorous, policy-driven memory governance to manage summarization drift, prevent attentional dilution, and support robust sequential decision-making in dynamic environments.

Introduction

Large Language Models have demonstrated exceptional computational linguistics capabilities, fundamentally altering the trajectory of artificial intelligence research. Beginning with text completion tasks, these models have progressively exhibited emergent abilities spanning codebase generation, logical deduction, and the capacity to receive natural language instructions in open-ended environments to invoke external tools[1]. However, deploying these models as autonomous agents exposes a critical architectural vulnerability: their inherent statelessness[2]. Traditional LLM inference treats each input independently, essentially resetting the model's internal cognitive state after every generation cycle. In long-horizon applications—such as personalized lifelong assistants, automated software engineering sprints, and scientific discovery platforms—agents must persist, organize, and selectively recall information across days, weeks, or thousands of interaction steps[3]. As task length and per-step complexity increase, the performance of unaugmented foundation models rapidly deteriorates, collapsing to near-zero accuracy on tasks exceeding extensive sequential steps without robust

persistent memory mechanisms[5].

The necessity for long-term consistency, adaptation to dynamic environments, and continual learning has driven the development of explicit, externalized memory mechanisms[2]. Memory in this context is defined not merely as a passive database lookup function, but as the agent's internal belief state within a Partially Observable Markov Decision Process (POMDP)[4]. Under this formalization, the agent utilizes memory to maintain a sufficient statistic of its interaction history, optimizing action selection subject to hard computational and storage budgets. Empirical ablation studies demonstrate that the gap between an agent equipped with an optimized memory architecture and one relying solely on base context window expansion is frequently larger than the performance delta between competing foundational models[4]. Memory transforms a static text generator into a genuinely adaptive entity capable of accumulating factual knowledge, developing behavioral patterns grounded in prior experience, avoiding the repetition of costly mistakes, and continuously evolving through interaction[3].

This report provides a comprehensive, systematic review of the state-of-the-art in memory and long-context mechanisms for LLM agents. The analysis is structured across a unified representation-management taxonomy that spans from base-model architectural modifications to high-level cognitive frameworks[10]. The investigation details foundational long-context techniques, analyzes the evolutionary stages of autonomous agent memory, explores multi-agent semantic infrastructure, and evaluates the modern benchmarking landscape. By organizing the field into coherent methodological families, the report delineates the theoretical and engineering principles required to construct reliable, long-horizon autonomous systems.

Methods

The research area of long-context mechanisms and agent memory can be organized into three primary operational domains: foundational context extension techniques within the base model, single-agent cognitive memory architectures, and multi-agent semantic infrastructures. This section systematically evaluates the literature across these domains.

Foundational Long-Context Modeling

Before invoking external memory modules, agents depend on the model's intrinsic ability to process extended token sequences, which constitutes the agent's "working memory" substrate[3]. Extending this capacity involves addressing the quadratic computational complexity of self-attention, the degradation of positional encodings over unprecedented lengths, and the exploding memory footprint of intermediate latent states during generation[12].

Length Extrapolation and Positional Embeddings

Modern transformer-based LLMs rely predominantly on Rotary Position Embedding (RoPE) to explicitly incorporate sequential positional information. RoPE divides query and key vectors into two-dimensional chunks and rotates them at specific frequencies[13]. However, when sequence lengths exceed those encountered during pre-training, RoPE fails to extrapolate gracefully, leading to catastrophic performance breakdown[13].

Historically, length generalization strategies were divided into two divergent paradigms. Out-of-Distribution (OOD) mitigation techniques, such as Position Interpolation (PI), NTK-aware scaling, YaRN, and LongRoPE, attempt to linearly or dynamically adjust the rotation frequencies to compress extended context coordinates back into the pre-trained distribution[13]. Conversely, semantic modeling approaches assert that as relative distances increase, standard rotation matrices inherently degrade the model's ability to discriminate between relevant and irrelevant tokens, motivating the use of higher base frequencies[13].

Recent theoretical analyses unify these perspectives by demonstrating mathematically that the low-frequency components of RoPE simultaneously govern OOD behavior under extrapolation and the stability of semantic attention over long contexts[13]. This insight has driven minimalist interventions such as Contextual Position Embedding (CoPE), which softly clips the low-frequency components of RoPE. CoPE eliminates OOD outliers and refines semantic signals without inducing the spectral leakage caused by hard clipping[13]. Other geometric interpretations, such as Polar Coordinate Position Embedding (PoPE), address the entanglement of content ("what") and position ("where") inherent in RoPE, eliminating the confound to improve independent matching decisions[17]. Furthermore, viewing the representation dimension as a time axis allows RoPE to be interpreted as a restricted wavelet transform using Haar-like wavelets; because it relies on a fixed scale parameter, it fails to exploit the multi-scale advantages of true wavelet transforms, explaining its poor inherent extrapolation[14]. To mitigate the high fine-tuning costs associated with optimizing these scaling factors for extreme lengths, frameworks like the Divide-and-Conquer Incremental Search (DCIS) algorithm have been proposed to strategically determine ideal scaling factors prior to fine-tuning, allowing models to generalize to long contexts while only training on short ones[16].

KV Cache Management and Compression

Even with perfectly extrapolated positional embeddings, the inference phase of long-context LLMs is critically bottlenecked by the Key-Value (KV) cache. The KV cache grows linearly with sequence length, consuming massive memory bandwidth and capacity during the autoregressive decode phase[18]. In continuous-inference streaming and long-horizon agentic workloads, the cache inevitably exceeds any fixed hardware budget. Consequently, KV cache eviction and compression have emerged as vital optimization vectors.

Early methods relied on heavy-tailed attention phenomena, recognizing that a small subset of tokens receives the majority of attention weights. StreamingLLM leverages the "attention sink" phenomenon, preserving the first few tokens—which disproportionately attract attention despite lacking semantic value—alongside a sliding window of recent tokens[18]. Methods like H2O and SnapKV accumulate historical attention scores at each decoding step, permanently evicting tokens that fall below a predefined threshold or retaining only the top-K "heavy hitters"[18].

While effective for reducing memory footprint, threshold-based and deterministic top-K eviction strategies suffer from inherent theoretical limitations. They convert transient marginality into permanent loss, deleting "bridge tokens" that may hold subtle, long-range

importance but lack immediate attention spikes[22]. Furthermore, static parameter thresholds fail to adapt dynamically to variable input complexities, meaning a threshold that preserves full performance on one dataset may cause catastrophic forgetting on another[18].

To overcome these constraints, the literature presents several advanced eviction paradigms. Table 1 summarizes the evolution of KV cache management strategies.

Eviction Strategy	Core Mechanism	Primary Advantage	Identified Limitations
Heuristic Windowing (e.g., StreamingLLM)	Retains attention sinks (initial tokens) plus a sliding window of recent tokens.	Highly efficient; requires no real-time attention sorting.	Discards crucial mid-document context entirely; fails on complex reasoning tasks.
Deterministic Top-K (e.g., H2O, SnapKV)	Accumulates historical attention scores; evicts tokens below a static threshold.	Retains semantically important "heavy hitters" across the entire sequence.	Vulnerable to threshold inflexibility; permanently erases subtly important bridge tokens.
Threshold-Free Budgeting (e.g., ReFreeKV)	Uses input-insensitive metrics based on the norm of the reduced attention matrix to dynamically adjust retention.	Adapts the compression ratio to the prompt's inherent difficulty.	Still relies on backward-looking attention statistics, introducing computational overhead.
Probabilistic Retention (e.g., Nexus Sampling)	Pairs iterative random walks over the direct attention graph with weighted reservoir sampling.	Maintains a non-zero inclusion probability for transient tokens, preventing permanent marginal erosion.	Requires maintaining parallel sampling structures during high-speed decode.
Policy-Learned Eviction (e.g., KV	Reframes eviction as an RL ranking	Attention-free at runtime; evaluates	Requires offline reinforcement

Policy - KVP)	task; trains per-head agents to predict future token utility.	based on future usefulness rather than past attention spikes.	learning over generation traces prior to deployment.

Specifically, policy-learned approaches such as KV Policy (KVP) represent a significant departure from heuristic compression. By training a distinct, lightweight RL agent for each KV head, KVP learns a specialized sorting policy using only key and value vectors. This avoids the computational overhead of repeatedly calculating historical attention matrices and ensures compatibility with hardware-optimized implementations like FlashAttention[21].

Linear Attention and State Space Models (SSMs)

To fundamentally bypass the quadratic computational complexity of self-attention and the linear memory scaling of the KV cache, structured State Space Models (SSMs)—most notably Mamba and RWKV—have gained traction as viable alternatives for long-horizon processing[24]. SSMs function as continuous-time systems discretized for sequences, maintaining a compact, constant-sized state that is updated at every step, thereby enabling linear-time training and static memory requirements during generation[25].

Architectures such as Falcon Mamba 7B demonstrate that pure SSM designs can achieve state-of-the-art performance, outperforming comparable transformer models on standard benchmarks while maintaining constant throughput regardless of context length[26]. To further reduce deployment overhead, extreme quantization efforts like Bi-Mamba have successfully binarized the Mamba architecture to 1-bit weights, drastically reducing energy consumption without catastrophic accuracy degradation[25]. Hybrid architectures seamlessly integrate linear-time sequence modeling with convolutional or transformer backbones to process high-resolution or ultra-long inputs efficiently[24].

Despite these computational advantages, rigorous empirical studies reveal that SSMs are inherently limited by a strong recency bias. While they manage long sequences efficiently, this bias impairs their ability to reliably recall distant information, necessitating deeper structures to facilitate true long-context reasoning[28]. This limitation underscores that architectural efficiency at the sequence level cannot substitute for dedicated, highly structured cognitive memory at the agent level.

Single-Agent Cognitive Memory Architectures

While context window extension provides the raw computational space for reasoning, a fully autonomous agent requires an overarching cognitive architecture capable of persisting, consolidating, and selectively recalling information over continuous multi-session interactions[3].

The Write-Manage-Read Loop

Agent memory is formalized as a continuous, tightly coupled loop of perception, deliberation, and action. Unlike standard database retrieval, the agent memory architecture is defined by

three interdependent operations operating continuously[3]:

1. **Write (Construction):** Determining what new observations, reflections, or tool outputs should enter the memory system, and formatting them into a structured schema (e.g., textual logs, vector embeddings, or knowledge graphs).
2. **Manage (Update):** The most frequently neglected but critical phase, involving the pruning, compression, consolidation, and deduplication of existing memories. This phase resolves contradictions, applies temporal decay, and prevents prompt pollution.
3. **Read (Query):** Retrieving relevant memories based on semantic intent, spatial relationships, or procedural applicability, and injecting them into the working context to guide action selection.

Because this loop is recursive, a single unmanaged write error can propagate downstream, polluting the belief state and degrading all subsequent decisions[4]. Consequently, memory mechanisms must deliberately navigate competing optimization objectives: utility (improving task outcomes), efficiency (token and latency costs), adaptivity (incremental updates without full retraining), faithfulness (avoiding stale or hallucinated recall), and governance (access control and privacy compliance)[4].

The Evolutionary Trajectory: Storage to Experience

The architectural development of agent memory can be categorized into three evolutionary stages, classified by the level of information abstraction and cognitive processing applied to the interaction trajectory[2]. A trajectory is defined as a chronological sequence of observation-action pairs within a task session.

Evolutionary Stage	Core Objective	Mechanism & Transformation	Example Implementations
Storage (Trajectory Preservation)	Resolve the contradiction between limited context and infinite interaction history.	Faithfully records raw observation-action traces with minimal transformation (linear logs, raw vector stores). Maintains a 1:1 mapping with history.	Basic Retrieval-Augmented Generation (RAG), episodic event logs, raw vector databases.
Reflection (Trajectory Refinement)	Rectify hallucinations, correct multi-step reasoning errors, and manage	Acts as a semantic filter. Analyzes completed trajectories through self-critique,	Self-questioning loops (ProMem), contradiction resolution systems, reflection modules

	memory lifecycles.	environment feedback, or introspection to generate refined, denoised memory units.	(Reflexion).
Experience (Trajectory Abstraction)	Enable continual learning and procedural mastery over long temporal horizons.	Cross-trajectory abstraction. Compresses redundant, topologically similar trajectories into generalized schemas, heuristics, or reusable executable skills, satisfying the Minimum Description Length principle.	Skill libraries (Voyager), learned procedural routines, generalized behavioral policies.

Systems trapped in the "Storage" phase exhibit predictable and severe failure modes. Context-resident memory systems relying on sliding windows or rolling summaries suffer from *summarization drift*; each sequential compression pass silently discards low-frequency details until the agent is left with a sanitized, generic version of history that fails catastrophically on edge cases[9]. Alternatively, relying solely on unmanaged Retrieval-Augmented Generation (RAG) vector stores leads to *memory blindness*, where semantic masking and embedding ambiguity cause the retrieval of stale records while highly relevant, slightly older facts are overlooked due to minor lexical mismatches[29].

To counter this, "Reflection" mechanisms introduce active epistemic feedback. For instance, the ProMem framework addresses "ahead-of-time" summarization limits—where an agent blindly summarizes history without knowing future tasks—by implementing a recurrent feedback loop[32]. Using self-questioning, the agent actively probes the dialogue history, recovering missing information and resolving contradictions to form high-fidelity supplementary memory[32].

Advanced Representation: Neuro-Symbolic and Code-Based Systems

As the field transitions toward the "Experience" stage, specialized memory representations have emerged to replace rudimentary vector stores. These advanced architectures structure

memory into explicitly defined temporal scopes: working memory (context window), episodic memory (concrete temporal experiences), semantic memory (abstracted, de-contextualized knowledge), and procedural memory (reusable skills and executable plans)[9].

User as Code (UaC): Traditional bag-of-facts retrieval struggles to aggregate data or enforce strict logical rules across scattered sessions. The UaC paradigm reframes memory as a living software project. An agent's model of a user or environment is stored as typed Python dataclasses, while the rules governing that state are stored as executable Python constraints[35]. The system extracts facts into an append-only log, periodically compiling them into structured code. This allows the memory to be queried via deterministic code execution—such as evaluating a boolean check on a medication list—rather than relying on probabilistic LLM semantic similarity. This paradigm perfectly bridges the gap between representation and verification, enabling the generation of active, unsolicited services based on state changes[35].

Neuro-Symbolic Memory Systems (NeuSymMS): Recognizing that neural networks excel at language interpretation but struggle with deterministic lifecycle management, NeuSymMS implements a hybrid architecture. Unstructured dialogue is processed by an LLM into atomic subject-relation-value triples. These facts are subsequently passed to a CLIPS-based expert system that enforces strict, rule-based policies for classification, deduplication, temporal decay, and access-based promotion across short- and long-term horizons. This architecture ensures auditable, contradiction-free memory updates suitable for highly regulated production environments[37].

Agent-Managed Zettelkasten Networks (A-MEM): Moving away from passive appended logs, A-MEM organizes memory as a dynamic knowledge network inspired by the Zettelkasten method[40]. When a new memory is created, the agent autonomously generates a comprehensive note with structured attributes, keywords, and explicit links to similar historical memories. The arrival of new information triggers recursive updates to the contextual representations of older memories, allowing the graph to organically refine its understanding and surface insights through simulated spreading activation[40].

Hierarchical Virtual Contexts: Systems like G-Memory trace extensive multi-agent interactions using a three-tier graph hierarchy encompassing insight, query, and interaction graphs. This allows bi-directional traversal to retrieve both high-level, cross-trial generalized insights and fine-grained interaction trajectories, updating the hierarchy dynamically upon task completion[44].

Policy-Learned and Parametric Memory Management

The absolute frontier of single-agent memory design replaces hard-coded management heuristics with learned policies. Heuristic strategies—such as fixed summarization schedules or generic k-NN retrieval—are rarely optimized for the specific end task. *Policy-learned management* systems frame memory operations (store, retrieve, update, discard) as callable tools or explicit actions within a reinforcement learning framework[4].

Systems like AgeMem and MemRL freeze the base LLM but optimize the retrieval and management policy using temporal difference learning or Proximal Policy Optimization. Under this paradigm, the agent discovers non-obvious tactics autonomously, such as proactively

summarizing intermediate results before the context window fills, or selectively discarding semantically similar but low-information records[4]. Metacognitive Memory Policy Optimization (MMPO) further refines this by utilizing Belief Entropy as an optimization proxy. Instead of relying solely on sparse outcome rewards, MMPO penalizes memory summaries that induce high epistemic uncertainty in the model, explicitly training the agent to maintain high-quality latent task states[46].

Taking policy evolution a step further, *parametric memory* frameworks like TMEM absorb distilled experiential supervision directly into fast, low-rank adaptation (LoRA) weights through lightweight online updates. Unlike non-parametric prompt space—where dropped context is permanently lost—parametric updates genuinely alter the agent's internal reasoning policy within a single episode, marrying rapid contextual lookup with internalized, permanent skill acquisition[4].

Multi-Agent Memory and Semantic Infrastructure

When agents scale from single-actor loops to complex, Multi-Agent Systems (MAS), memory ceases to be an individual cognitive trait and becomes a shared organizational infrastructure[44]. Multi-agent memory introduces profound challenges in synchronization, concurrent access control, and information overload[47].

In shared environments, multiple principals write to a common memory pool but query it under varying scopes, roles, and security clearances. If unmanaged, this results in memory homogenization—a lack of role-aware customization—and chaotic prompt pollution[48]. Solutions like Collaborative Memory implement dynamic access control, separating private fragments visible only to the originating agent from selectively shared organizational insights[48]. Similarly, architectures like LatentMem customize latent memory entries dynamically based on role-aware parameters, significantly reducing information overload across the collective[48].

To formalize decentralized communication without requiring a central database coordinator or exposing proprietary model weights, the Mesh Memory Protocol (MMP) provides a foundational semantic infrastructure for multi-agent LLM systems[34]. MMP shifts interaction from simple message passing to true cognitive collaboration through four integrated primitives:

1. **CAT7 Schema:** A fixed seven-field schema structuring every Cognitive Memory Block (CMB), enabling heterogeneous agents spanning different platforms to parse shared cognitive state uniformly[34].
2. **Receiver-Autonomous SVAF (Semantic Value Assessment Function):** Rather than accepting whole messages blindly, each receiving agent evaluates incoming CMB fields through its own role-indexed fusion gate. This makes the act of writing to memory an evaluative, receiver-controlled act rather than a passive append[34].
3. **Lineage Tracking:** Parent and ancestor content-hash keys track provenance across the network. This prevents echo-chamber loops, allowing agents to ground claims and reliably recognize returning ideas as derivatives of their own prior reasoning[34].
4. **Remix:** Newly evaluated intersections of domain knowledge are packaged into remixes, creating a navigable Directed Acyclic Graph (DAG) of collaborative memory that survives

session restarts[34].

Crucially, MMP guarantees *observation-only inference*; agents relax local free energies based on incoming typed observations, converging on a globally optimal conclusion (identification-complete) without ever transmitting their internal parameters, gradients, or raw hidden states, thereby ensuring enterprise-grade confidentiality[49].

Evaluation Paradigms and Benchmarks

As memory architectures have evolved from passive storage to active management, evaluation methodologies have undergone a concurrent, necessary paradigm shift. Historically, models were evaluated on perplexity or static recall; modern evaluation demands dynamic, multi-session, interactive metrics that assess reasoning endurance[3]. Table 2 outlines the progression of long-context and memory evaluation frameworks.

Evaluation Paradigm	Focus	Representative Benchmarks	Core Findings
Static / Synthetic Recall	Base context window integrity and simple retrieval.	NIAH, MNIAH-R, DENIAHL, Visual Haystacks, U-NIAH	Uncovered the "lost-in-the-middle" effect; proved that data features (patterns, type) heavily influence recall; revealed RAG noise sensitivity.
Realistic Document Tasks	Complex integration across authentic, massive documents.	LongBench (v1, v2, Pro), Loong, ∞-Bench	Human-level performance requires multi-hop reasoning; partial vs. full dependency tasks expose differing architectural weaknesses.
Agentic / Multi-Session	Memory-Action-Environment loops over extended horizons.	LORE, MemoryArena, EvoMemBench, LoCoMo	Models with perfect static recall fail when memory must guide future actions; reasoning endurance

			collapses exponentially with step count.

Static and Synthetic Long-Context Evaluation

The foundational unit test for long-context capability is the Needle-In-A-Haystack (NIAH) benchmark, which embeds a specific fact within a large volume of distractor text[31]. While standard NIAH successfully identified the "lost-in-the-middle" phenomenon, models quickly saturated single-fact retrieval. This led to advanced variants such as Multiple Needles In A Haystack Reasoning (MNIAH-R), which demands multi-hop reasoning across several dispersed facts, and Visual Haystacks (VHs), which tests true multi-image spatial integration rather than mere Optical Character Recognition[55].

Detailed data-centric evaluations, such as the DENIAHL benchmark, ablate factors beyond mere context length. DENIAHL demonstrates that retrieval accuracy is highly dependent on intrinsic data features like patterns, data types, and item length. This indicates that even models with virtually infinite context windows suffer catastrophic degradation when searching for non-patterned, numeric, or lexically ambiguous needles[57]. The U-NIAH framework further reveals that advanced reasoning models occasionally exhibit reduced RAG compatibility due to hypersensitivity to semantic distractors, resulting in induced self-doubt or hallucinations when conflicting noise is present[59].

Realistic and Multitask Document Benchmarks

Moving beyond synthetic needles, real-world benchmarks evaluate context capabilities over authentic documents. LongBench and its subsequent iterations compile tasks requiring deep integration across documents up to 2 million tokens in length, including financial reports, long-dialogue histories, and repository-level code comprehension[60]. To address scalability and quality, LongBench Pro utilizes a Human-Model Collaborative Construction pipeline, leveraging frontier LLMs to draft challenging questions and design rationales, which are subsequently verified by human experts. These benchmarks categorize difficulty based on whether a task requires *partial* dependency (localized retrieval) or *full* dependency (integrating distant, scattered spans)[63]. The Loong benchmark similarly stress-tests multi-document scenarios where omitting any single document guarantees task failure, requiring advanced Spotlighting, Comparison, Clustering, and Chain of Reasoning tasks[64].

Agentic and Multi-Session Benchmarking

The most critical gap identified in modern evaluation is that passive recall does not guarantee active utility. Agents that achieve near-perfect scores on static long-context datasets frequently fail when required to utilize memory to inform sequential actions[65].

To bridge this, the field has introduced dynamic, POMDP-based evaluation environments.

- **LORE (Long-horizon Reasoning Evaluation):** A rule-based, controllable platform that generates arbitrarily extendable chains of logical dependencies. LORE isolates reasoning

endurance from superficial API complexity, revealing that state-of-the-art agents' accuracy collapses entirely beyond 120 steps due to cascading error accumulation[6].

- **MemoryArena:** A unified evaluation gym specifically targeting the Memory-Agent-Environment loop. Agents engage in interdependent multi-session tasks—such as preference-constrained group travel planning or bundled web shopping—where they must distill experiences from early sessions into memory and utilize those insights to resolve constraints in later tasks. This heavily penalizes systems that passively log data without structural integration[65].
- **EvoMemBench:** Analyzes memory from a self-evolving perspective across two axes: temporal scope (in-episode vs. cross-episode evolution) and cognitive content (knowledge-oriented vs. execution-oriented). The benchmark demonstrates that retrieval-based memory excels at knowledge retention, but procedural or state-based memory is absolutely required for execution-oriented multi-step tasks[69].

Challenges and Prospects

Despite rapid architectural innovation across the stack, the deployment of long-horizon LLM agents faces several persistent, interlocking challenges that define the near-term research frontier.

The Retrieval vs. Parametric Integration Gap: There is a fundamental, unresolved tension between non-parametric external stores and parametric weight adaptations. External stores (vector databases, knowledge graphs) offer auditable, privacy-compliant, and highly interpretable memory that supports targeted deletion. However, they suffer from inherent retrieval bottlenecks, semantic masking, and latency[9]. Parametric updates (e.g., continuous LoRA adjustments) seamlessly integrate knowledge into the model's fundamental reasoning pathways, drastically reducing retrieval latency and improving holistic understanding. Yet, parametric memory currently lacks robust mathematical mechanisms for unlearning, surgical forgetting, or temporal conflict resolution[9]. Future hybrid models must rigorously reconcile these paradigms to achieve both seamless reasoning and verifiable compliance.

Summarization Drift and Epistemic Decay: In autonomous systems running continuously for weeks or months, the reliance on rolling summaries or heuristic pruning introduces profound epistemic decay. High-importance but low-frequency constraints (e.g., "never modify the production schema directly") are often silently omitted during aggressive compression cycles[9]. Managing the delicate balance between context window efficiency and high-fidelity factual preservation requires the development of self-correcting maintenance protocols, akin to the active self-questioning seen in recurrent feedback loops[32].

Memory Governance and Mnemonic Sovereignty: As agent memory becomes a centralized, actionable repository of organizational and personal intelligence, security paradigms must shift from traditional training-data leakage concerns to active memory-lifecycle governance. Agents with persistent writable memory are highly vulnerable to cross-session memory poisoning, where adversaries inject malicious instructions via query-only interactions that corrupt long-term behavior and propagate across multi-agent shared state[46]. Ensuring "mnemonic sovereignty"—defined as verifiable, recoverable governance over what may be written, who

may read, when updates are authorized, and which states must be permanently forgotten—requires integrating traditional operating-system access control logic (such as that seen in NeuSymMS[38]) directly into probabilistic LLM workflows.

Causally Grounded Retrieval: Current retrieval mechanisms primarily index by semantic similarity, lexical overlap, or temporal recency. However, sophisticated sequential decision-making relies on causality. An agent navigating a complex state space needs to retrieve the historical action that *caused* a specific error, not merely the text most semantically similar to the current prompt[3]. Developing embedding models and dependency-structured graphs (such as ContextWeaver[71]) that capture causal and logical relationships natively remains an open, critical frontier for the field.

Conclusion

The evolution of memory in Large Language Models represents a fundamental paradigm shift from stateless pattern matching to stateful, autonomous cognition. Achieving long-horizon robustness requires a synthesis across the entire computational and cognitive stack. At the foundation, mechanisms like CoPE, dynamic KV cache eviction, and State Space Models optimize the processing of extended sequences. However, raw context length is emphatically not a substitute for architectural memory. The literature decisively indicates that effective long-term agents must transition from basic storage to structured experience, relying on explicit, policy-driven write-manage-read loops to curate, compress, and consolidate their interaction histories.

Frameworks such as User as Code, neuro-symbolic rule systems, and policy-learned eviction strategies demonstrate that memory must be dynamically managed, auditable, and inherently verifiable. Furthermore, as agents begin to operate collectively, protocols like the Mesh Memory Protocol provide the necessary semantic infrastructure for secure, decentralized knowledge integration without compromising proprietary internals. Moving forward, the development of causally grounded retrieval, robust mnemonic sovereignty, and parametric-non-parametric hybrids will dictate the viability of LLM agents in unbounded, real-world deployments. Benchmarking suites have evolved to demand not just passive recall, but the active utilization of memory to drive future action, setting a rigorous standard for the next generation of artificial intelligence.

引用的著作

1. Agent Harness for Large Language Model Agents: A Survey[v3] | Preprints.org, https://www.preprints.org/manuscript/202604.0428
2. From Storage to Experience: A Survey on the Evolution of LLM Agent Memory Mechanisms, https://www.preprints.org/manuscript/202601.0618/v1
3. [2603.07670] Memory for Autonomous LLM Agents:Mechanisms, Evaluation, and Emerging Frontiers - arXiv, https://arxiv.org/abs/2603.07670
4. Memory for Autonomous LLM Agents:Mechanisms, Evaluation, and Emerging Frontiers - arXiv, https://arxiv.org/pdf/2603.07670
5. The Agent's Marathon: Probing the Limits of Endurance in Long-Horizon Tasks |

OpenReview, https://openreview.net/forum?id=dAn82lpLx4
6. THE AGENT'S MARATHON: PROBING THE LIMITS OF ENDURANCE IN LONG-HORIZON TASKS - OpenReview, https://openreview.net/pdf?id=dAn82lpLx4
7. [2605.06716] From Storage to Experience: A Survey on the Evolution of LLM Agent Memory Mechanisms - arXiv, https://arxiv.org/abs/2605.06716
8. From Storage to Experience: A Survey on the Evolution of LLM Agent Memory Mechanisms, https://www.preprints.org/manuscript/202601.0618
9. Memory for Autonomous LLM Agents: Mechanisms, Evaluation, and Emerging Frontiers, https://arxiv.org/html/2603.07670v1
10. LLM Agent Memory: A Survey from a Unified Representation–Management Perspective, https://www.preprints.org/manuscript/202603.0359/v1
11. LLM Agent Memory: A Survey from a Unified Representation--Management Perspective, https://openreview.net/forum?id=E6LJBlfoPL
12. [2503.17407] A Comprehensive Survey on Long Context Language Modeling - arXiv, https://arxiv.org/abs/2503.17407
13. CoPE: Clipped RoPE as A Scalable Free Lunch for Long Context LLMs - arXiv, https://arxiv.org/pdf/2602.05258
14. arXiv:2502.02004v1 [cs.CL] 4 Feb 2025, https://arxiv.org/pdf/2502.02004
15. Frayed RoPE and Long Inputs: A Geometric Perspective - ResearchGate, https://www.researchgate.net/publication/402859294_Frayed_RoPE_and_Long_Inputs_A_Geometric_Perspective
16. DCIS: Efficient Length Extrapolation of LLMs via Divide-and-Conquer Scaling Factor Search - arXiv, https://arxiv.org/pdf/2412.18811
17. Decoupling the “What” and “Where” with Polar Coordinate Positional Embedding - arXiv, https://arxiv.org/html/2509.10534
18. ReFreeKV: Towards Threshold-Free KV Cache Compression - arXiv, https://arxiv.org/html/2502.16886v4
19. RocketKV: Accelerating Long-Context LLM Inference via Two-Stage KV Cache Compression - arXiv, https://arxiv.org/html/2502.14051v3
20. KV Cache Compression and Its Infra Problems - Research at NVIDIA, https://research.nvidia.com/labs/eai/blogs/kv-cache-compression-and-its-infra-problems/
21. Learning to Evict from Key-Value Cache - arXiv, https://arxiv.org/html/2602.10238
22. Forget Without Compromise: Nexus Sampling for Streaming KV-Cache Eviction Under Fixed Budgets - arXiv, https://arxiv.org/html/2606.23961v1
23. Meta-Soft: Leveraging Composable Meta-Tokens for Context-Preserving KV Cache Compression - arXiv, https://arxiv.org/html/2605.22337v2
24. Mamba or RWKV: Exploring High-Quality and High-Efficiency Segment Anything Model - arXiv, https://arxiv.org/html/2406.19369v1
25. Bi-Mamba: Towards Accurate 1-Bit State Space Models - arXiv, https://arxiv.org/html/2411.11843v1
26. Falcon Mamba: The First Competitive Attention-free 7B Language Model - arXiv, https://arxiv.org/html/2410.05355v1
27. StateSpaceDiffuser: Bringing Long Context to Diffusion World Models - NIPS,

https://papers.neurips.cc/paper_files/paper/2025/file/63943ee9fe347f3d95892cf87d9a42e6-Paper-Conference.pdf
28. Understanding and Mitigating Bottlenecks of State Space Models through the Lens of Recency and Over-smoothing - arXiv, https://arxiv.org/html/2501.00658v2
29. A Practical Guide to Memory for Autonomous LLM Agents | Towards Data Science, https://towardsdatascience.com/a-practical-guide-to-memory-for-autonomous-llm-agents/
30. From Storage to Experience: A Survey on the Evolution of LLM Agent Memory Mechanisms* - ACL Anthology, https://aclanthology.org/2026.findings-acl.2069.pdf
31. The Quest for Needles in the Haystack: Why LLM Evaluation Needs This Test - Chris Zhang, https://zhanghaolin66.medium.com/the-quest-for-needles-in-the-haystack-why-llm-evaluation-needs-this-test-1c75ba4bd917
32. Beyond Static Summarization: Proactive Memory Extraction for LLM Agents - arXiv, https://arxiv.org/html/2601.04463v1
33. What Are Agentic Workflows? Patterns, Memory and Examples - Addepto, https://addepto.com/blog/what-are-agentic-workflows-patterns-memory-and-examples/
34. Mesh Memory Protocol: Semantic Infrastructure for Multi-Agent LLM Systems - arXiv, https://arxiv.org/html/2604.19540v1
35. User as Code: Executable Memory for Personalized Agents - arXiv, https://arxiv.org/html/2606.16707v1
36. User as Code: Executable Memory for Personalized Agents - arXiv, https://arxiv.org/pdf/2606.16707
37. NeuSymMS: A Hybrid Neuro-Symbolic Memory System for Persistent, Self-Curating LLM Agents - arXiv, https://arxiv.org/html/2605.17596v1
38. NeuSymMS: A Hybrid Neuro-Symbolic Memory System for LLM Agents - arXiv, https://arxiv.org/pdf/2605.17596
39. [2605.17596] NeuSymMS: A Hybrid Neuro-Symbolic Memory System for Persistent, Self-Curating LLM Agents - arXiv, https://arxiv.org/abs/2605.17596
40. NeurIPS Poster A-Mem: Agentic Memory for LLM Agents, https://neurips.cc/virtual/2025/poster/119020
41. A-Mem: Agentic Memory for LLM Agents - arXiv, https://arxiv.org/html/2502.12110v11
42. A-MEM: Zettelkasten for agents | Alpha's Manifesto, https://blog.alphasmanifesto.com/2026/04/11/a-mem-zettelkasten-for-agents/
43. GitHub - WujiangXu/A-mem: The code for NeurIPS 2025 paper "A-Mem: Agentic Memory for LLM Agents", https://github.com/WujiangXu/A-mem
44. Tracing Hierarchical Memory for Multi-Agent Systems - NIPS, https://papers.neurips.cc/paper_files/paper/2025/file/136a45cd9b841bf785625709a19c6508-Paper-Conference.pdf
45. Self-Evolving Agents: Real Learning, or Memory in a Costume? | by Micheal Lanham,

https://medium.com/@Micheal-Lanham/self-evolving-agents-real-learning-or-memory-in-a-costume-c397f46bbfce
46. Daily Papers - Hugging Face, https://huggingface.co/papers?q=Memory-augmented%20LLM%20agents
47. Memory in LLM-based Multi-agent Systems: Mechanisms, Challenges, and Collective Intelligence | TechRxiv, https://www.techrxiv.org/doi/10.36227/techrxiv.176539617.79044553
48. Daily Papers - Hugging Face, https://huggingface.co/papers?q=multi-principal%20shared-memory%20agents
49. Mesh Inference A Formal Model of Collective Intelligence Without a Center - arXiv, https://arxiv.org/html/2606.19537v1
50. [2604.19540] Mesh Memory Protocol: Semantic Infrastructure for Multi-Agent LLM Systems, https://arxiv.org/abs/2604.19540
51. [2604.03955] Symbolic-Vector Attention Fusion for Collective Intelligence - arXiv, https://arxiv.org/abs/2604.03955
52. Mesh Inference: A Formal Model of Collective Intelligence Without a Center - arXiv, https://arxiv.org/pdf/2606.19537
53. A Survey on Evaluation of LLM-based Agents - arXiv, https://arxiv.org/html/2503.16416v2
54. Evaluation and Benchmarking of LLM Agents: A Survey - arXiv, https://arxiv.org/html/2507.21504v1
55. Reasoning on Multiple Needles In A Haystack - arXiv, https://arxiv.org/html/2504.04150v1
56. Visual Haystacks: A Vision-Centric Needle-In-A-Haystack Benchmark - arXiv, https://arxiv.org/html/2407.13766v3
57. DENIAHL: In-Context Features Influence LLM Needle-In-A-Haystack Abilities - arXiv, https://arxiv.org/html/2411.19360v1
58. DENIAHL: In-Context Features Influence LLM Needle-In-A-Haystack Abilities - arXiv, https://arxiv.org/abs/2411.19360
59. U-NIAH: Unified RAG and LLM Evaluation for Long Context Needle-In-A-Haystack - arXiv, https://arxiv.org/abs/2503.00353
60. -LongBench: Are de facto Long-Context Benchmarks Literally Evaluating Long-Context Ability? - arXiv, https://arxiv.org/html/2505.19293
61. [R] LongBench v2: Towards Deeper Understanding and Reasoning on Realistic Long-context Multitasks - Reddit, https://www.reddit.com/r/MachineLearning/comments/1hwfs48/r_longbench_v2_towards_deeper_understanding_and/
62. [2412.15204] LongBench v2: Towards Deeper Understanding and Reasoning on Realistic Long-context Multitasks - arXiv, https://arxiv.org/abs/2412.15204
63. LongBench Pro: A More Realistic and Comprehensive Bilingual Long-Context Evaluation Benchmark - arXiv, https://arxiv.org/html/2601.02872v1
64. Leave No Document Behind: Benchmarking Long-Context LLMs with Extended Multi-Doc QA - arXiv, https://arxiv.org/html/2406.17419
65. MemoryArena: Benchmarking Agent Memory in Interdependent Multi-Session Agentic Tasks - arXiv, https://arxiv.org/pdf/2602.16313

66. [2602.16313] MemoryArena: Benchmarking Agent Memory in Interdependent Multi-Session Agentic Tasks - arXiv, https://arxiv.org/abs/2602.16313
67. Benchmarking Agent Memory in Interdependent Multi-Session Agentic Tasks - arXiv, https://arxiv.org/html/2602.16313v1
68. MemoryArena: Benchmarking Agent Memory in Interdependent Multi-Session Agentic Tasks, https://memoryarena.github.io/
69. EvoMemBench: Benchmarking Agent Memory from a Self-Evolving Perspective - arXiv, https://arxiv.org/html/2605.18421v2
70. From Storage to Experience: A Survey on the Evolution of LLM Agent Memory Mechanisms, https://www.researchgate.net/publication/399892293_From_Storage_to_Experience_A_Survey_on_the_Evolution_of_LLM_Agent_Memory_Mechanisms
71. ContextWeaver: Selective and Dependency-Structured Memory Construction for LLM Agents - ResearchGate, https://www.researchgate.net/publication/404248128_ContextWeaver_Selective_and_Dependency-Structured_Memory_Construction_for_LLM_Agents